ENCONTRO 1 DE 6

Do C ao binário

Curso Introdutório de Programação de Baixo Nível

Liga ProtoFPGA · UFSC Araranguá

```c
int main(void) {
    int a     = 12;
    int b     = 5;
    int soma  = a + b;
    int dobro = soma + soma;
    return 0;
}
```

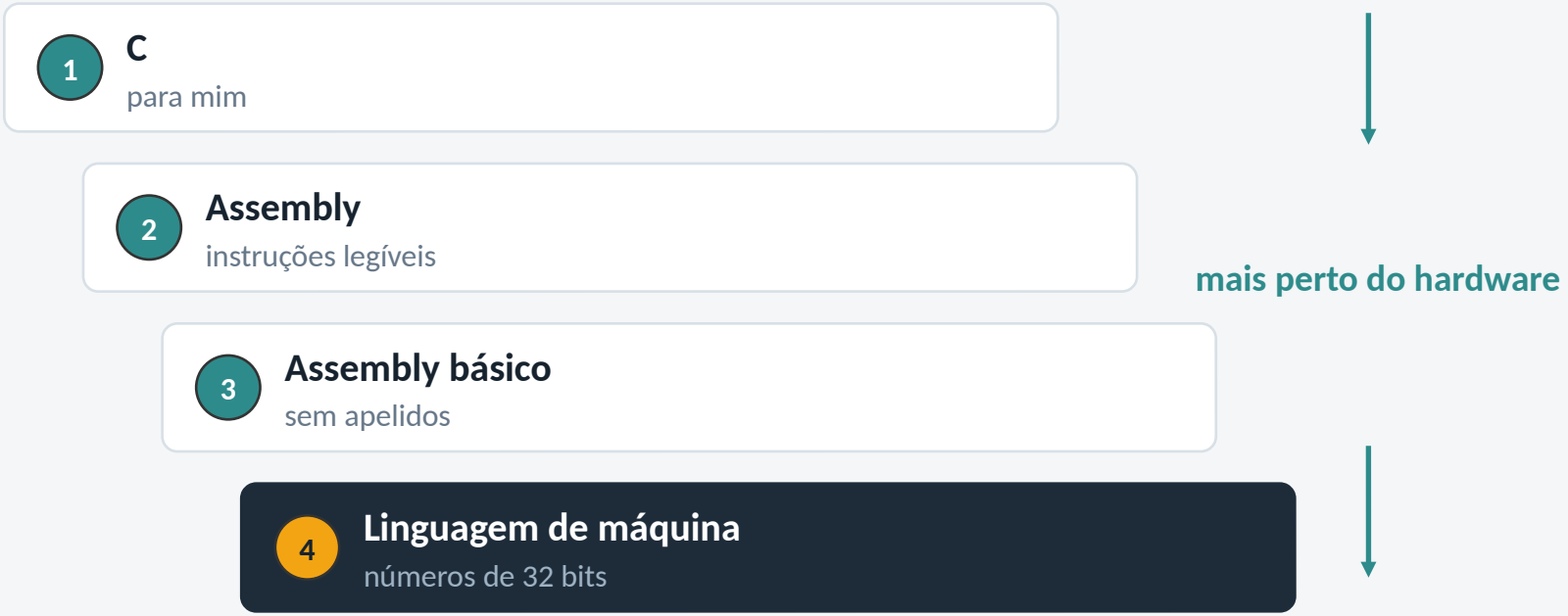
O processador não sabe ler isto.

Ele não sabe o que é int, nem o que é “soma”. Ele lida com números.

Então como isto faz alguma coisa acontecer?

2 · OS QUATRO DEGRAUS

Não é um pulo. São degraus.



É o mesmo programa, escrito de quatro jeitos. Vamos descer um degrau de cada vez.

Quem faz cada tradução

1

Compilador

de C

para Assembly

2

Montador

de Assembly

para número

3

Processador

lê o número

e executa

No MARS, o montador é o botão Assemble.

C: você escreve a intenção

```
int a     = 12;
int b     = 5;
int soma  = a + b;
int dobro = soma + soma;
```

O que o C não conta:

- Onde cada valor fica guardado
- Quantas operações a máquina faz
- Em que ordem elas acontecem

Isso é uma vantagem: escrever fica rápido e o programa fica legível.

É também o motivo de o processador não conseguir executar isto direto.

Nomes como “soma” existem para você. Eles não chegam ao hardware.

Assembly: uma operação por linha

```
main:
    li   $t0, 12          # a = 12
    li   $t1, 5           # b = 5
    add  $t2, $t0, $t1    # soma  = a + b
    add  $t3, $t2, $t2    # dobro = soma + soma
```

$t0, $t1... registradores: gavetas dentro do processador

main: um rótulo — uma etiqueta com nome, dada a um ponto do programa

Primeiro vem o destino

```
add   $t2,   $t0,   $t1
```

$t2
destino
recebe o resultado

$t0
origem
lido, não alterado

$t1
origem
lido, não alterado

```
$t2   ←   $t0  +  $t1
```

Ler não apaga: depois da soma, $t0 e $t1 continuam com os mesmos valores.

O que o montador resolve

o que eu escrevo

```
main:
  li  $t0, 12
  li  $t1, 5
  add $t2,$t0,$t1
```


o que o montador entende

```
  addiu $8,$0,12
  addiu $9,$0,5
  add   $10,$8,$9
```

`main:` virou um endereço

`li` virou addiu

`$t0` virou $8

`.text  # comentário` não vira nada

8 · DUAS PALAVRAS NOVAS

Pseudoinstrução e diretiva

Pseudoinstrução

Abreviação que o montador aceita e troca por instrução(ões) real(is).

```
li $t0, 12    →    addiu $8, $0, 12
```

Diretiva

Recado para o montador. Não vira instrução nenhuma.

```
.text        .globl main
```

$0 é o registrador que vale sempre zero. Somar 12 com zero dá 12.

A instrução é um número

```
add  $t2, $t0, $t1
```

↓

0x01095020

0000 0001 0000 1001 0101 0000 0010 0000

32 bits. Sempre 32, para qualquer instrução.

Ninguém decora isto. Basta perceber: o que eu leio também é um número.

Os quatro degraus, na mesma tela

Address	Code	Basic	Source
0x00400000	0x2408000c	addiu $8,$0,12	li $t0, 12
0x00400008	0x01095020	add $10,$8,$9	add $t2,$t0,$t1

endereço | degrau 4 | degrau 3 | degrau 2

Da direita para a esquerda, a mesma instrução vai ficando menos humana e mais máquina.

Se na coluna Basic aparecer 0x0000000c no lugar de 12: Settings → desmarcar "Values displayed in hexadecimal".

O MARS não é o processador

Simulador (MARS)

- Programa que imita um processador
- Dá para parar no meio
- Dá para voltar atrás
- Dá para olhar dentro

Processador de verdade

- Circuito físico
- Executa e pronto
- Não dá para pausar
- É o que veremos no encontro 6

Editou o arquivo? Assemble de novo — senão o MARS executa a versão antiga.

Cada instrução muda um registrador

1	`li $t0, 12`	`$t0: 0 → 12`
2	`li $t1, 5`	`$t1: 0 → 5`
3	`add $t2, $t0, $t1`	`$t2: 0 → 17`
4	`add $t3, $t2, $t2`	`$t3: 0 → 34`

Dois exercícios

1 Quais os valores no final?

```
li   $t0, 7
li   $t1, 3
add  $t2, $t0, $t1
add  $t3, $t2, $t0
```

$t2 = ?** **$t3 = ?

2 Trocando 12 por 20

```
li   $t0, 12
li   $t0, 20
```

Quantas linhas da coluna Code mudam?

nenhuma · uma · todas

Atividade

1. **Observar** — abrir o aula1_soma.asm e montar
2. **Prever** — escrever os valores antes de executar
3. **Modificar** — mudar um valor e prever de novo
4. **Explicar** — uma frase sobre o que a instrução fez

O enunciado completo está na folha de atividade.

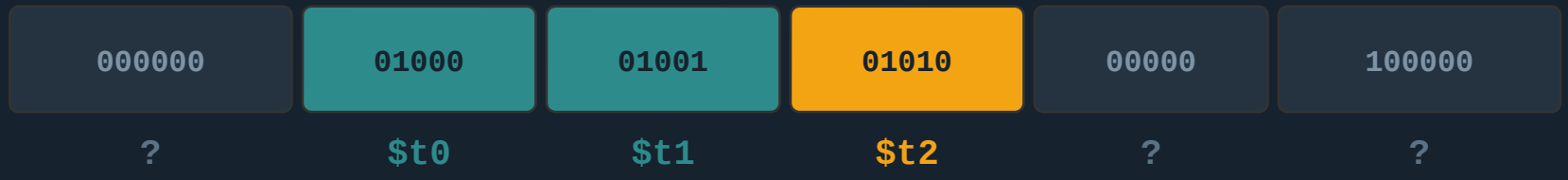
O que fica de hoje

1. C é conveniente para o humano. O processador não lê C.
2. Assembly mostra as operações uma a uma.
3. O montador transforma cada instrução num número de 32 bits.
4. O processador executa esses números — e o registrador muda de valor.

Nomes de variáveis não existem no hardware. Eles somem no caminho.

Esse número não é aleatório

```
add  $t2, $t0, $t1
```



Os três registradores estão ali dentro, escritos em binário. O resto ainda é ponto de interrogação.

Na aula 2, a gente abre o número inteiro.